Studio Nx

S1-11b WAV 출력 — 함수 설명 합본 (JS 297~300)

2026-10-20

목차

1. ai-notes/javascript/S1-11b - WAV 출력/297_audio_wav.md
2. ai-notes/javascript/S1-11b - WAV 출력/298_audio_export_plan.md
3. ai-notes/javascript/S1-11b - WAV 출력/299_audio_offline.md
4. ai-notes/javascript/S1-11b - WAV 출력/300_sequencer_export.md

JavaScript 297 — WAV 인코딩 (`audio/wav.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/wav.js`
작업	S1-11b WAV 출력: 렌더링한 소리를 WAV 파일 바이트로 바꿉니다 (FR-303)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 296 화면 연결, 다음 298 내보내기 계획 계산

1. 하는 일

채널별 `Float32Array`(값은 -1에서 1)를 받아 WAV 파일의 바이트(`Uint8Array`)로 만듭니다. 16bit, 24bit, 32bit float를 지원하고, 정규화에 쓸 피크 계산과 배율 계산도 여기에 있습니다. 소리 장치를 전혀 쓰지 않는 순수 함수라서 Node에서도 시험합니다.

```
렌더링 결과 [왼쪽 Float32Array, 오른쪽 Float32Array]
        │  peakOfChannels ─▶ 피크 ─▶ normalizeGain ─▶ 배율 (정규화를 켰을 때만)
        ▼
   encodeWav(채널들, 샘플레이트, 비트 깊이, { gain, dither })
        ▼
   [RIFF 머리][fmt 조각][(fact 조각)][data 조각: 프레임마다 왼쪽, 오른쪽 번갈아]
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `startExport` (300)	렌더링이 끝나면 피크를 재고, 정규화 배율을 구하고, WAV 바이트를 만들어 다운로드에 넘깁니다.
`wavByteLength`	내보내기 창의 "예상 파일 크기"를 렌더링 전에 계산합니다.
시험	`export_unit.mjs`가 머리와 값을, `render_offline.mjs`가 브라우저의 디코더로 다시 읽은 값을 확인합니다.

3. 코드

3.1 피크와 정규화 배율

```
export function peakOfChannels(channels) {
  let peak = 0;
  for (let c = 0; c < channels.length; c++) {
    const data = channels[c];
    for (let i = 0; i < data.length; i++) {
      const v = data[i] < 0 ? -data[i] : data[i];
      if (v > peak) peak = v;
    }
  }
```

```
  return peak;
}
```

```
export function normalizeGain(peak, targetDb) {
  if (!(peak > 0)) return 1;
  return Math.pow(10, targetDb / 20) / peak;
}
```

3.2 파일 크기

```
export function wavByteLength(frames, numChannels, bitDepth) {
  const header = bitDepth === 32 ? 58 : 44;   // 32bit float 는 fmt 확장(2바이트)과 fact 조각(12바이트)이
더 있음
  return header + frames * numChannels * (bitDepth / 8);
}
```

3.3 인코딩

```
export function encodeWav(channels, sampleRate, bitDepth, options) {
  const opt = options || {};
  const gain = opt.gain != null ? opt.gain : 1;
  const dither = opt.dither == null ? true : !!opt.dither;
  if (BIT_DEPTHS.indexOf(bitDepth) < 0) throw new Error('지원하지 않는 비트 깊이: ' + bitDepth);
  const numCh = channels.length;
  const frames = numCh > 0 ? channels[0].length : 0;
  const bytes = bitDepth / 8;
  const isFloat = bitDepth === 32;
  const total = wavByteLength(frames, numCh, bitDepth);
  const buffer = new ArrayBuffer(total);
  const view = new DataView(buffer);

  // 머리
  let p = 0;
  writeText(view, p, 'RIFF'); view.setUint32(p + 4, total - 8, true); writeText(view, p + 8, 'WAVE'); p
+= 12;
  writeText(view, p, 'fmt '); view.setUint32(p + 4, isFloat ? 18 : 16, true);
  view.setUint16(p + 8, isFloat ? 3 : 1, true);                  // 1 = PCM, 3 = IEEE float
  view.setUint16(p + 10, numCh, true);
  view.setUint32(p + 12, sampleRate, true);
  view.setUint32(p + 16, sampleRate * numCh * bytes, true);      // 초당 바이트
  view.setUint16(p + 20, numCh * bytes, true);                   // 한 프레임의 바이트
  view.setUint16(p + 22, bitDepth, true);
  p += 24;
  if (isFloat) {
    view.setUint16(p, 0, true); p += 2;                          // 확장 크기 0
    writeText(view, p, 'fact'); view.setUint32(p + 4, 4, true); view.setUint32(p + 8, frames, true); p +
= 12;
  }
  writeText(view, p, 'data'); view.setUint32(p + 4, frames * numCh * bytes, true); p += 8;

  // 데이터 (채널을 프레임마다 번갈아 씀)
  let seed = 22222;                                              // 디더 난수는 고정 시드: 같은 입력이면
같은 파일
  const rand = function () { seed = (seed * 1664525 + 1013904223) >>> 0; return seed / 4294967296; };
```

```
  for (let i = 0; i < frames; i++) {
    for (let c = 0; c < numCh; c++) {
      const v = channels[c][i] * gain;
      if (isFloat) {
        view.setFloat32(p, v, true); p += 4;
      } else if (bitDepth === 16) {
        let s = v * 32767;
        if (dither) s += rand() - rand();                        // TPDF: 최대 ±1 LSB
        s = Math.round(s);
        if (s > 32767) s = 32767; else if (s < -32768) s = -32768;
        view.setInt16(p, s, true); p += 2;
      } else {
        let s = Math.round(v * 8388607);
        if (s > 8388607) s = 8388607; else if (s < -8388608) s = -8388608;
        if (s < 0) s += 16777216;                                // 24bit 음수는 2의 보수
        view.setUint8(p, s & 255); view.setUint8(p + 1, (s >> 8) & 255); view.setUint8(p + 2, (s >> 16)
& 255); p += 3;
      }
    }
  }
  return new Uint8Array(buffer);
}
```

4. WAV 파일의 모양

위치(바이트)	내용	16bit와 24bit	32bit float
0 ~ 11	`RIFF`, 파일 크기 - 8, `WAVE`	같음	같음
12 ~ 35	`fmt ` 조각: 형식, 채널, 샘플레이트, 초당 바이트, 프레임 바이트, 비트	형식 1(PCM), 조각 크기 16	형식 3(IEEE float), 조각 크기 18
이어서	`fact` 조각: 프레임 수	없음	있음 (12바이트)
이어서	`data` 조각: 소리	머리 44바이트	머리 58바이트

한 프레임은 채널 수만큼의 샘플이 이어진 것입니다. 스테레오는 왼쪽, 오른쪽, 왼쪽, 오른쪽 순서로 씁니다.

5. 최신 문법으로 쓰면

```
// peakOfChannels 와 normalizeGain 을 최신 문법으로 쓴 모양
export const peakOfChannels = (channels) =>
  channels.reduce((peak, data) => data.reduce((p, v) => Math.max(p, Math.abs(v)), peak), 0);

export const normalizeGain = (peak, targetDb) =>
  peak > 0 ? 10 ** (targetDb / 20) / peak : 1;

// encodeWav 의 옵션 읽기
export function encodeWav(channels, sampleRate, bitDepth, { gain = 1, dither = true } = {}) { /* ... */
}
```

전통 문법 (실제 코드)	최신 문법
겹친 `for` 문으로 최댓값 찾기	`reduce` 두 번
`Math.pow(10, x)`	`10 ** x`

전통 문법 (실제 코드)	최신 문법
`opt.gain != null ? opt.gain : 1`	기본값 구조 분해 `{ gain = 1 }`
`function () { ... }`	`() => ...`

6. 왜 이렇게

선택	이유
정규화는 배율 `gain`만 넘기고 채널은 건드리지 않습니다	몇 분짜리 곡은 채널 배열이 수백 MB입니다. 정규화하려고 배열을 한 번 더 복사하지 않고, 파일에 쓰는 순간 곱합니다.
정규화 배율은 렌더링 뒤의 피크로 구합니다	이펙트 체인을 거친 최종 소리의 가장 큰 값이 목표 dBFS가 되도록 하는 것이 정규화의 정의입니다. 소리가 전혀 없으면(피크 0) 배율을 1로 두어 0으로 나누지 않습니다.
16bit에는 TPDF 디더를 넣습니다	32bit float로 계산한 소리를 16bit로 줄이면 작은 소리의 끝이 계단처럼 거칠어집니다(양자화 잡음). 최대 ±1 LSB의 아주 작은 잡음을 섞으면 이 거친 느낌이 부드러운 잡음으로 바뀝니다. FL Studio도 16bit 출력에 디더 옵션을 둡니다.
디더의 난수는 고정 시드의 선형 합동 생성기입니다	같은 소리를 두 번 내보내면 같은 파일이 나와야 시험하기 쉽고, 문제가 생겼을 때 재현할 수 있습니다. 소리가 거칠어지지 않는다는 목적에는 시드가 무엇이든 상관없습니다.
24bit와 32bit에는 디더를 넣지 않습니다	24bit는 양자화 잡음이 이미 들리지 않을 만큼 작고, 32bit float는 줄이는 과정이 없습니다.
32bit는 정수가 아니라 부동소수점(IEEE float)입니다	내부 계산과 같은 형식이라 값이 그대로 남고, 1을 넘는 값도 잘리지 않습니다. 이어서 믹싱이나 마스터링을 할 때 쓰기 좋습니다. FL Studio도 같은 이유로 32bit float를 둡니다.
32bit 머리에는 `fact` 조각과 확장 크기를 넣습니다	표준은 PCM이 아닌 형식에 이 두 가지를 요구합니다. 일부 프로그램은 없으면 파일을 열지 못합니다. 브라우저의 디코더는 둘 다 읽는 것을 시험에서 확인했습니다.
정수 값은 범위를 잘라서(클리핑) 씁니다	마스터가 0dBFS를 넘으면 값이 정수 범위를 벗어납니다. 잘라 두지 않으면 부호가 뒤집혀 큰 잡음이 납니다.
24bit는 바이트 세 개를 직접 씁니다	`DataView`에는 24bit 쓰기 함수가 없습니다. 음수는 2의 보수(16777216을 더한 값)로 바꿔 아래쪽 바이트부터 씁니다.
파일 크기를 `wavByteLength`로 따로 둡니다	같은 계산을 인코더와 창의 크기 안내가 함께 쓰므로, 안내한 크기와 실제 크기가 어긋나지 않습니다. 시험에서 둘이 같은 것을 확인합니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 WAV 부분(38건)이 머리와 값을 확인합니다. 16bit의 길이, `RIFF`와 `WAVE`와 `data` 글자, 파일 크기 필드, 형식 번호(1), 채널과 샘플레이트와 비트 필드, 초당 바이트와 프레임 바이트, 0.5가 16384, 1.0이 32767인지 봅니다. 2와 -2는 32767과 -32768로 잘립니다. 디더는 같은 입력이면 같은 파일이고, 끄면 값이 한결같으며, 켜면 값이 흔들리되 2 LSB 안에 있습니다. 24bit는 0.5가 4194304이고, 32bit float는 형식 번호 3과 `fact` 조각을 갖고 값이 그대로이며 1.5도 그대로입니다.
- `docs/handoff/s1-11b_tests/render_offline.mjs`의 J 부분(11건)은 실제 렌더링 결과를 세 가지 비트 깊이로 인코딩하고, 브라우저의 `decodeAudioData`로 다시 읽어서 길이, 샘플레이트, 채널 수, 파일 크기, 값을 비교합니다. 16bit는 최대 차이가 0.0002 이하, 24bit는 0.000001 이하, 32bit는 거의 0입니다. 정규화를 켜면 피크가 -0.3dBFS(0.966)가 됩니다.

- 시험 중 발견한 일: 처음에 두 건이 실패했습니다. -0.5를 16bit로 바꾼 값이 -16384가 아니라 -16383이었습니다. 자바스크립트의 `Math.round`가 정확히 .5인 값을 항상 위쪽(+방향)으로 올리기 때문입니다. 소리에는 영향이 없는 1 LSB 차이라서 코드는 두고, 시험의 기대값을 고치고 이유를 주석으로 남겼습니다.

JavaScript 298 — 내보내기 계획 계산 (`audio/export_plan.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/export_plan.js`
작업	S1-11b WAV 출력: 창에서 고른 옵션을 "어디서부터 어디까지, 꼬리는 몇 초, 파일명은 무엇"으로 바꿉니다 (FR-301, FR-307)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 297 WAV 인코딩, 다음 299 오프라인 렌더링

1. 하는 일

내보내기 창의 옵션을 숫자로 바꾸는 순수 함수 모음입니다. 화면이나 소리 장치를 쓰지 않아서 Node에서 시험합니다.

함수	입력, 출력
`projectEndTick`	모든 클립 중 가장 뒤에 끝나는 tick (곡의 끝)
`resolveExportRange`	범위 모드(전체, 루프, 선택한 클립, 직접 지정)와 현재 상태를 받아 시작 tick과 끝 tick을 돌려줍니다. 만들 수 없으면 이유를 돌려줍니다.
`rangeSeconds`	범위의 길이(초). 템포 변화를 반영합니다.
`autoTailSeconds`	꼬리 "Auto" 값: 이펙트와 악기가 범위 끝 뒤로 남기는 소리의 길이
`sanitizeFileName`, `exportFileName`	파일명 (`{프로젝트명}_{YYYYMMDD_HHmm}`)
`formatDuration`, `formatBytes`	창에 보여 줄 `1:05`, `11.8 MB` 같은 글자

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `refreshForm`과 `openExport` (300)	범위별 설명, 요약 줄, 기본 파일명, 꼬리 Auto 값을 채웁니다.
`sequencer-export.js`의 `startExport` (300)	렌더링에 넘길 시작 tick과 끝 tick을 구합니다.

3. 코드

3.1 곡의 끝과 범위

```
export function projectEndTick(clips) {
  let end = 0;
  (clips || []).forEach(function (c) {
    const e = c.start_tick + c.length_tick;
    if (e > end) end = e;
  });
  return end;
}
```

```
export function resolveExportRange(opt) {
  const tpb = opt.ticksPerBar;
  if (opt.mode === 'all') {
    const end = projectEndTick(opt.clips);
    return end > 0 ? { ok: true, startTick: 0, endTick: end } : { ok: false, reason: 'empty' };
  }
  if (opt.mode === 'loop') {
    const loop = opt.loop;
    if (!loop || !(loop.end > loop.start)) return { ok: false, reason: 'no-loop' };
    return { ok: true, startTick: loop.start, endTick: loop.end };
  }
  if (opt.mode === 'selected') {
    const sel = opt.selected || [];
    if (sel.length === 0) return { ok: false, reason: 'no-selection' };
    let start = Infinity;
    sel.forEach(function (c) { if (c.start_tick < start) start = c.start_tick; });
    const end = projectEndTick(sel);
    return end > start ? { ok: true, startTick: start, endTick: end } : { ok: false, reason: 'no-selecti
on' };
  }
  if (opt.mode === 'custom') {
    const from = Math.floor(Number(opt.fromBar));
    const to = Math.floor(Number(opt.toBar));
    if (!(from >= 1) || !(to >= from)) return { ok: false, reason: 'bad-range' };
    return { ok: true, startTick: (from - 1) * tpb, endTick: to * tpb };
  }
  return { ok: false, reason: 'bad-range' };
}
```

```
export function rangeSeconds(startTick, endTick, baseBpm, changes) {
  return ticksToSecondsWithChanges(endTick, baseBpm, changes) - ticksToSecondsWithChanges(startTick, bas
eBpm, changes);
}
```

3.2 꼬리 Auto

```
export function autoTailSeconds(effects, layers, bpm) {
  let tail = 0;
  (effects || []).forEach(function (e) {
    if (e.is_bypassed) return;
    const p = resolveParams(e.schema, e.params);
    if (e.plugin_code === 'reverb') {
      tail = Math.max(tail, (Number(p.decay) || 2) + (Number(p.pre_delay) || 0));
    } else if (e.plugin_code === 'delay') {
      const fb = Math.max(0, Math.min(0.95, Number(p.feedback) || 0));
      const repeats = fb > 0 ? Math.min(60, Math.ceil(Math.log(0.001) / Math.log(fb))) : 1;
      tail = Math.max(tail, delaySeconds(p, bpm) * (repeats + 1));
    }
  });
  (layers || []).forEach(function (l) {
    if (l.is_enabled === false) return;
    const p = resolveParams(l.schema, l.params);
    if (p.env_release != null) tail = Math.max(tail, Number(p.env_release) || 0);
  });
```

```
  return Math.min(MAX_TAIL_SECONDS, Math.ceil(tail * 2) / 2);
}
```

3.3 파일명과 표시

```
export function sanitizeFileName(name) {
  const cleaned = String(name == null ? '' : name).replace(/[\\/:*?"<>|\u0000-\u001f]/g, '_').replace(/
\s+/g, ' ').trim();
  return cleaned.replace(/^\.+/, '').slice(0, 80) || 'Studio Nx';
}
```

```
export function exportFileName(title, date) {
  const two = function (n) { return (n < 10 ? '0' : '') + n; };
  const stamp = date.getFullYear() + two(date.getMonth() + 1) + two(date.getDate()) + '_' + two(date.get
Hours()) + two(date.getMinutes());
  return sanitizeFileName(title) + '_' + stamp;
}
```

```
export function formatDuration(seconds) {
  const s = Math.max(0, Math.round(seconds));
  const m = Math.floor(s / 60);
  const r = s % 60;
  return m + ':' + (r < 10 ? '0' : '') + r;
}
```

```
export function formatBytes(bytes) {
  if (bytes < 1024 * 1024) return Math.max(1, Math.round(bytes / 1024)) + ' KB';
  return (bytes / (1024 * 1024)).toFixed(1) + ' MB';
}
```

4. 범위 모드

모드	시작	끝	못 만드는 경우 (reason)
`all` 전체	0	가장 뒤 클립의 끝	클립이 하나도 없으면 `empty`
`loop` 루프 구간	루프 시작	루프 끝	루프가 꺼져 있으면 `no-loop`
`selected` 선택한 클립	선택한 클립 중 가장 앞 시작	선택한 클립 중 가장 뒤 끝	선택이 없으면 `no-selection`
`custom` 직접 지정	`fromBar` 마디의 처음	`toBar` 마디의 끝 (그 마디도 포함)	마디 번호가 1 미만이거나 끝이 시작보다 앞이거나 숫자가 아니면 `bad-range`

마디 번호는 화면과 같이 1부터 셉니다. 3마디부터 4마디까지는 `(3 - 1) × 3840`에서 `4 × 3840` tick입니다. 같은 마디(3에서 3)는 한 마디 길이가 됩니다.

5. 최신 문법으로 쓰면

```
// projectEndTick 과 sanitizeFileName 을 최신 문법으로 쓴 모양
export const projectEndTick = (clips = []) =>
  Math.max(0, ...clips.map((c) => c.start_tick + c.length_tick));
```

```
export const sanitizeFileName = (name) =>
  String(name ?? '').replace(/[\\/:*?"<>|\u0000-\u001f]/g, '_').replace(/\s+/g, ' ').trim()
    .replace(/^\.+/, '').slice(0, 80) || 'Studio Nx';

// exportFileName 의 두 자리 맞춤
const two = (n) => String(n).padStart(2, '0');
```

전통 문법 (실제 코드)	최신 문법
`forEach`로 최댓값 갱신	`Math.max(...배열)` (클립이 수만 개면 스프레드는 위험해서 실제 코드는 반복문)
`name == null ? '' : name`	`name ?? ''`
`(n < 10 ? '0' : '') + n`	`String(n).padStart(2, '0')`

6. 왜 이렇게

선택	이유
"전체"는 0에서 가장 뒤 클립의 끝까지입니다	FL Studio와 Ableton Live도 곡의 처음부터 마지막 클립 끝까지 내보냅니다. 마커나 눈금은 곡의 끝으로 보지 않습니다(화면 위의 마커가 35마디까지 있어도 클립이 6마디에서 끝나면 6마디까지 내보냅니다).
선택한 클립의 범위는 가장 앞 시작에서 가장 뒤 끝까지입니다	클립 사이에 빈 구간이 있어도 그 시간 관계를 유지하려는 것입니다. 선택하지 않은 클립의 소리는 `offline.js`에서 뺍니다.
직접 지정의 끝 마디는 포함합니다	사용자가 "9마디부터 25마디"라고 쓰면 25마디의 소리까지 듣고 싶다는 뜻이기 때문입니다.
못 만드는 경우를 예외가 아니라 `{ ok: false, reason }`으로 돌려줍니다	창이 열려 있는 동안 입력이 바뀔 때마다 부르는 함수입니다. 입력 중에는 잘못된 값이 흔하므로 오류로 끊지 않고, 이유를 보여 주고 내보내기 버튼만 끕니다.
꼬리 Auto는 효과별로 가장 긴 값을 고릅니다	리버브는 `decay + pre_delay`, 딜레이는 되먹임이 -60dB로 줄 때까지의 반복(`ln 0.001 / ln 되먹임`번)에 딜레이 시간을 곱한 값, 악기는 가장 긴 `env_release`입니다. 바이패스한 이펙트와 꺼진 악기 층은 뺍니다. 효과가 겹치는 정확한 합이 아니라 넉넉한 어림값입니다.
꼬리는 0.5초 단위로 올리고 30초에서 자릅니다	어림값이 모자라서 꼬리가 잘리는 쪽보다 조금 남는 쪽이 낫습니다. 30초는 창의 입력 한도와 같고, 되먹임 0.95에 4초 딜레이 같은 극단 값이 무한히 늘어나는 것을 막습니다.
파일명에서 `₩ / : * ? " < >` 와 제어 문자를 `_`로 바꿉니다	
파일명 시각은 지역 시간입니다	사용자가 보는 시계와 같아야 파일 목록에서 찾기 쉽습니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 계획 부분(37건)이 확인합니다. 곡 끝은 33마디이고 클립이 없으면 0입니다. 전체, 루프, 선택, 직접 지정(9마디 처음부터 25마디 끝, 같은 마디는 한 마디 길이)의 범위가 맞고, 빈 곡과 루프 없음과 선택 없음과 잘못된 범위는 각각 이유를 돌려줍니다. 120BPM에서 한 마디는 2초입니다. 꼬리는 리버브 decay 2에 pre_delay 0.1이면 2.5초, 바이패스하면 0, 딜레이는 시간 0.5초에 되먹임 0.5이면 5초대, 되먹임 0이면 1초, 악기 릴리스 1.2초는 1.5초이고 극단 값도 30초를 넘지 않습니다. 파일명은 금지 글자와 공백과 앞의 점을 정리하고, 한글은 그대로 두며, 시각은 `20261020_1405`와 `20270103_0407`처럼 0을 채웁니다. 시간 표시와 크기 표시도 확인합니다.
- `docs/handoff/s1-11b_tests/ui_export.mjs`는 실제 화면에서 범위 설명(전체 `bar 1 – 6`, 선택한 클립 `bar 5 – 6`, 직접 지정 `bar 3 – 4`)과 파일명 형식을 확인합니다.

JavaScript 299 — 오프라인 렌더링 (`audio/offline.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/offline.js`, `studionx/frontend/static/js/audio/mixer.js` (`applyTracks`의 `immediate`)
작업	S1-11b WAV 출력: 재생과 같은 그래프를 `OfflineAudioContext`에 만들어 범위의 소리를 계산합니다 (FR-302, FR-306)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 298 내보내기 계획 계산, 다음 300 내보내기 창

1. 하는 일

범위 안의 소리를 실시간보다 빠르게 계산해서 돌려줍니다. 재생(290)이 쓰는 `MixerGraph`, `EffectRack`, `InstrumentRack`, `ClipPlayer`, `NotePlayer`를 그대로 쓰고, 시계만 실시간 `AudioContext`에서 `OfflineAudioContext`로 바꿉니다. 그래서 들리는 소리와 파일의 소리가 같은 코드에서 나옵니다.

```
renderOffline(옵션)
  1. 길이 계산: 범위(초) + 꼬리  ─▶  new OfflineAudioContext(2, 프레임 수, 샘플레이트)
  2. 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 맞춰짐)                    진행률 0~10%
  3. 믹서, 이펙트, 악기, 클립 재생기를 재생과 똑같이 만들고 트랙 값을 즉시 반영
  4. 2초 창마다 멈춰서(suspend) 그 창의 소리만 예약하고 다시 진행(resume)     진행률 10~100%
  5. startRendering ─▶ 채널별 Float32Array
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `startExport` (300)	창의 옵션과 화면 스냅샷을 넘겨 렌더링을 시킵니다. 진행률과 취소 확인 함수도 함께 넘깁니다.
시험 `render_offline.mjs`	클립, 노트, 이펙트, 믹서, 범위, 꼬리, 진행률, 취소를 숫자로 확인합니다.

3. 코드

3.1 순수 함수

```
export function clipsInRange(clips, startTick, endTick) {
  const out = [];
  (clips || []).forEach(function (c) {
    const end = c.start_tick + c.length_tick;
    if (end <= startTick || c.start_tick >= endTick) return;
    const copy = {};
    Object.keys(c).forEach(function (k) { copy[k] = c[k]; });
    copy.length_tick = Math.min(end, endTick) - c.start_tick;
    out.push(copy);
  });
```

```
  return out;
}
```

```
export function windowBoundaries(startTick, endTick, baseBpm, changes, windowSec) {
  const t0 = ticksToSecondsWithChanges(startTick, baseBpm, changes);
  const total = ticksToSecondsWithChanges(endTick, baseBpm, changes) - t0;
  const out = [startTick];
  for (let k = 1; k * windowSec < total; k++) {
    const tick = Math.ceil(secondsToTicks(t0 + k * windowSec, baseBpm, changes));
    if (tick > out[out.length - 1] && tick < endTick) out.push(tick);
  }
  out.push(endTick);
  return out;
}
```

3.2 렌더링

```
export async function renderOffline(o) {
  const changes = o.tempoChanges || [];
  const progress = o.onProgress || function () {};
  const cancelled = o.isCancelled || function () { return false; };
  const Ctx = o.OfflineContext || (typeof OfflineAudioContext !== 'undefined' ? OfflineAudioContext : nu
ll);
  if (!Ctx) throw new Error('이 브라우저는 OfflineAudioContext 를 지원하지 않습니다');

  const rangeSec = ticksToSecondsWithChanges(o.endTick, o.bpm, changes) - ticksToSecondsWithChanges(o.st
artTick, o.bpm, changes);
  const totalSec = rangeSec + Math.max(0, o.tailSec || 0);
  const frames = Math.max(1, Math.ceil(totalSec * o.sampleRate));
  const ctx = new Ctx(2, frames, o.sampleRate);

  const audioClips = clipsInRange(o.audioClips, o.startTick, o.endTick);
  const midiClips = clipsInRange(o.midiClips, o.startTick, o.endTick);

  // 1) 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 다시 맞춰짐). 전체 진행률의 앞 10%
  const buffers = {};
  const ids = [];
  audioClips.forEach(function (c) { if (c.sample_id != null && ids.indexOf(String(c.sample_id)) < 0) id
s.push(String(c.sample_id)); });
  for (let i = 0; i < ids.length; i++) {
    if (cancelled()) throw new Error('cancelled');
    try {
      const bytes = await o.loadSampleBytes(Number(ids[i]));
      buffers[ids[i]] = await ctx.decodeAudioData(bytes);
    } catch (err) {
      if (typeof console !== 'undefined') console.warn('[export] 샘플 ' + ids[i] + ' 을(를) 불러오지 못해
건너뜁니다', err);
    }
    progress(0.1 * (i + 1) / ids.length);
  }

  // 2) 재생과 같은 그래프
  const mixer = new MixerGraph(ctx);
  const effects = new EffectRack(ctx, mixer, o.getEffects, function () { return o.bpm; });
  const rack = new InstrumentRack(ctx, function (id) { return mixer.trackInput(id); }, o.getInstrument
```

```
s);
  const player = new ClipPlayer(ctx, function (id) { return mixer.trackInput(id); }, function (sid) { re
turn buffers[String(sid)] || null; });
  const notePlayer = new NotePlayer(ctx, function (id, pitch, vel, when, dur) { return rack.playNote(id,
pitch, vel, when, dur); });
  effects.sync(o.tracks);
  mixer.applyTracks(o.tracks, true);

  // 3) 창마다 소리를 예약
  const noteEvents = expandNotes(midiClips).filter(function (ev) { return ev.abs_start >= o.startTick;
});
  const bounds = windowBoundaries(o.startTick, o.endTick, o.bpm, changes, WINDOW_SECONDS);
  const scheduleWindow = function (k) {
    const a = bounds[k];
    const b = bounds[k + 1];
    const nowTime = ticksToSecondsWithChanges(a, o.bpm, changes) - ticksToSecondsWithChanges(o.startTic
k, o.bpm, changes);
    if (k === 0) {
      // 시작 위치에 걸쳐 있는 클립은 중간부터 울림
      clipsCovering(audioClips, a).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, change
s); });
    }
    clipsStartingIn(audioClips, a, b).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, chang
es); });
    notesStartingIn(noteEvents, a, b).forEach(function (ev) { notePlayer.schedule(ev, a, nowTime, o.bpm,
changes); });
  };
  scheduleWindow(0);

  // 4) 창 경계마다 멈춰서 다음 창을 예약. 경계 시각은 렌더링 샘플 단위로 맞춰짐
  const t0 = ticksToSecondsWithChanges(o.startTick, o.bpm, changes);
  const stops = [];
  for (let k = 1; k < bounds.length - 1; k++) {
    const sec = ticksToSecondsWithChanges(bounds[k], o.bpm, changes) - t0;
    if (sec > 0 && sec < totalSec) stops.push({ k: k, sec: sec });
  }
  let failed = null;
  stops.forEach(function (s) {
    ctx.suspend(s.sec).then(function () {
      if (failed) { ctx.resume(); return; }
      if (cancelled()) { failed = new Error('cancelled'); ctx.resume(); return; }   // 남은 구간은 소리
없이 빠르게 끝내고 결과를 버림
      scheduleWindow(s.k);
      progress(0.1 + 0.9 * (s.sec / totalSec));
      ctx.resume();
    });
  });

  const rendered = await ctx.startRendering();
  if (failed) throw failed;
  progress(1);
  const channels = [];
  for (let c = 0; c < rendered.numberOfChannels; c++) channels.push(rendered.getChannelData(c));
  return { channels: channels, sampleRate: rendered.sampleRate, frames: rendered.length };
}
```

3.3 믹서의 즉시 반영 (`mixer.js`)

```
applyTracks(tracks, immediate) {
  const audible = {};
  audibleTrackIds(tracks).forEach(function (id) { audible[String(id)] = true; });
  const self = this;
  const now = this.context.currentTime;
  // 재생 중에는 뚝 끊기는 소리(클릭)를 막으려고 아주 짧게 이어서 바꾸고, 렌더링 시작 때(immediate)는 바로
그 값으로 둠
  // (뮤트한 트랙이 처음 몇십 ms 동안 새어 나오지 않게 하려고)
  const set = function (param, value) {
    if (immediate) param.setValueAtTime(value, now);
    else param.setTargetAtTime(value, now, 0.01);
  };
  (tracks || []).forEach(function (t) {
    if (t.track_type === 'MASTER') {
      set(self.master.gain, dbToGain(t.volume_db));
      return;
    }
    const s = self.strip(t.track_id);
    const gain = audible[String(t.track_id)] ? dbToGain(t.volume_db) : 0;
    set(s.input.gain, gain);
    if (s.pan) set(s.pan.pan, Math.max(-1, Math.min(1, Number(t.pan) || 0)));
  });
}
```

4. 시간 창 예약

모든 노드를 한꺼번에 만들지 않고, 2초 길이의 창마다 그 창에서 시작하는 소리만 만듭니다.

```
시간(초):   0 ———————— 2 ———————— 4 ———————— 6 — ...
창:        [  창 0    )[  창 1    )[  창 2    )
예약 시점:  렌더링 전    2초에 멈춤   4초에 멈춤      각 지점에서 suspend → 다음 창 예약 → resume
```

경계	처리
창의 경계 tick	`windowBoundaries`가 구합니다. 인접한 창이 같은 경계 값을 쓰므로 소리가 빠지거나 두 번 만들어지지 않습니다.
경계를 올림합니다	경계 tick에서 시작하는 소리가 창이 열리는 시각(`suspend`한 시각)보다 이르면 안 되기 때문입니다. 반올림하면 최대 반 tick(약 0.26ms) 일찍 시작할 수 있습니다.
시작 위치에 걸친 클립	첫 창에서 `clipsCovering`으로 찾아 중간부터 울립니다.
범위 시작 전에 시작한 노트	울리지 않습니다. 실시간 재생이 중간부터 재생할 때 그 노트를 울리지 않는 규칙과 같습니다.
범위 끝을 넘는 클립과 노트	`clipsInRange`가 끝에서 자릅니다. 노트의 릴리스와 이펙트의 꼬리는 꼬리 구간에서 자연스럽게 사라집니다.

5. 최신 문법으로 쓰면

```
// clipsInRange 를 최신 문법으로 쓴 모양
export const clipsInRange = (clips = [], startTick, endTick) =>
```

```
  clips
    .filter((c) => c.start_tick + c.length_tick > startTick && c.start_tick < endTick)
    .map((c) => ({ ...c, length_tick: Math.min(c.start_tick + c.length_tick, endTick) - c.start_tick
}));

// 샘플 디코딩을 병렬로 (실제 코드는 진행률을 알리려고 순서대로 기다립니다)
const entries = await Promise.all(ids.map(async (id) => [id, await ctx.decodeAudioData(await loadSampleB
ytes(id))]));
```

전통 문법 (실제 코드)	최신 문법
`Object.keys(c).forEach`로 복사	객체 전개 `{ ...c }`
`for` 문과 `await`로 하나씩 디코딩	`Promise.all`로 한꺼번에
`o.tailSec || 0`	`o.tailSec ?? 0`

6. 왜 이렇게

선택	이유
재생과 같은 클래스를 그대로 씁니다	내보낸 파일이 화면에서 듣던 소리와 다르면 안 됩니다. 소리를 만드는 코드가 하나이면 둘이 어긋날 수 없습니다. S1-11a에서 모든 클래스가 컨텍스트를 인자로 받게 만든 이유입니다.
창마다 멈춰서 노드를 만듭니다	곡 전체의 노드를 한 번에 만들면 노트가 수천 개일 때 메모리를 많이 쓰고 시작이 느립니다. 또 `startRendering`은 한 번 시작하면 끝까지 가기 때문에, 중간에 진행률을 알리거나 멈출 방법이 `suspend`뿐입니다.
창의 길이는 2초입니다	너무 짧으면 멈추는 횟수가 늘어 느려지고, 너무 길면 진행률이 뚝뚝 끊기고 취소 반응이 느립니다. 12초 곡이면 5번 알립니다.
샘플을 오프라인 컨텍스트로 디코딩합니다	`decodeAudioData`는 디코딩한 소리를 컨텍스트의 샘플레이트로 다시 맞춰 줍니다. 44.1kHz 샘플을 48kHz로 내보내도 음정이 바뀌지 않는다는 것을 시험(440Hz)으로 확인했습니다.
렌더링 시작 때 트랙 값은 즉시 반영합니다 (`immediate`)	실시간 재생은 클릭 소리를 막으려고 10ms에 걸쳐 값을 옮깁니다. 렌더링은 시각 0에서 시작하므로, 이 방식이면 뮤트한 트랙도 처음 수십 ms 동안 새어 나옵니다. 시작할 때는 바로 그 값으로 두어야 합니다. 시험으로 뮤트한 트랙이 처음부터 끝까지 완전한 무음인 것을 확인합니다.
취소하면 남은 구간을 빠르게 끝내고 결과를 버립니다	`OfflineAudioContext`는 도중에 닫을 수 없습니다. 멈춘 채로 두면 `startRendering`이 영원히 끝나지 않으므로, 다음 창의 소리를 더 만들지 않고 `resume`해서 빠르게 마친 뒤 `cancelled` 오류로 알립니다. 취소 반응은 최대 한 창(2초) 늦을 수 있습니다.
샘플 하나를 못 불러와도 나머지를 내보냅니다	한 파일 때문에 곡 전체를 포기하는 것보다, 그 클립만 빠지고 경고를 남기는 편이 낫습니다. 재생(290)도 같은 방식입니다.
꼬리는 계산할 길이에만 더합니다	범위의 끝 뒤에는 새 소리를 예약하지 않고, 이미 울리고 있던 이펙트 꼬리와 노트 릴리스가 사라질 시간만 줍니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 마지막 부분(10건)은 순수 함수를 확인합니다. `clipsInRange`는 범위와 겹치는 클립만 남기고, 끝은 범위 끝에서 자르며, 원본은 바꾸지 않고, 끝과 시작이 정확히 맞닿은 클립은 뺍니다. `windowBoundaries`는 경계가 오름차순이고 중복이 없으며 처음과 끝이 범위와 같고, 짧은 범위는 한 창이며, 템포가 바뀌어도 끝을 넘지 않습니다.
- `docs/handoff/s1-11b_tests/render_offline.mjs` (46건)는 실제 `OfflineAudioContext`로 소리를 그려 숫자로 확인합니다.
    - A: 2초 클립이 88200 프레임, 피크 0.5, 좌우 같음, 440Hz입니다.

    - B: 뮤트한 트랙은 처음부터 완전한 무음, 트랙 볼륨 -6dB와 마스터 -6dB는 피크가 절반, 팬 왼쪽 끝은 오른쪽이 무음, 솔로 트랙(880Hz)만 들립니다.
    - C: 48kHz면 96000 프레임이고 음정은 같습니다.
    - D: 범위 시작 위치에 따라 클립이 중간부터 울리고, 범위 끝에서 소리가 잘리며, 꼬리 구간은 무음이고, 범위 밖의 클립은 소리가 없습니다.
    - E: Synth 노트가 C4(261.6Hz)로 울리고, 여러 시간 창(0초, 2초, 4초)의 노트가 모두 울리며, 시작 전에 시작한 노트는 울리지 않고, 끝을 넘는 노트는 잘립니다.
    - F: 12초 클립에서 모든 0.1초 구간에 소리가 있습니다. 창 경계에서 끊기지 않습니다.
    - G: 진행률은 5번 이상 줄지 않고 알려 주며 마지막이 1입니다. 취소하면 `cancelled` 오류입니다.
    - H: 딜레이와 리버브의 꼬리가 범위 끝 뒤에 남고, 이펙트가 없으면 꼬리 구간은 무음입니다.
    - I: 60BPM이면 1920tick이 2초이고, 템포 변경도 반영합니다.
    - J: WAV로 내보낸 뒤 브라우저가 다시 읽은 값이 같습니다 (297에서 설명).
- 시험 중 발견한 일: 템포 변경 시험이 한 번 실패했습니다. 120BPM에서 1초 뒤 240BPM이 되는 곡의 길이를 1.25초로 계산했는데, 1920tick은 240BPM에서 0.5초이므로 1.5초가 맞습니다. 렌더링 결과(1.5초)가 옳았고 시험의 계산이 틀렸습니다.

JavaScript 300 — 내보내기 창 (`sequencer-export.js`, `studioNx.html`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-export.js`, `studionx/frontend/static/js/pages/sequencer-audio.js` (`exportSnapshot`와 선택 표시), `studionx/frontend/studioNx.html` (`#export-modal`), `studionx/frontend/static/css/pages/studio.css` (`.ex-*`)
작업	S1-11b WAV 출력: 내보내기 창과 다운로드 (FR-301, FR-306, FR-307)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 299 오프라인 렌더링, 다음 없음 (S1-11b의 마지막)
화면 목업	`docs/page_mockups/export_modal/` (6장)

1. 하는 일

FILE 메뉴의 Export..., 상단의 ↓ 버튼, 단축키 Ctrl+Shift+R이 여는 창입니다. 설정, 진행 중, 완료 세 화면으로 바뀝니다. 소리 계산은 `audio/` 폴더(297, 298, 299)가 하고, 이 파일은 화면의 값을 읽고 진행률을 보여 주고 파일을 내려받게 하는 일만 합니다.

```
창 열기(openExport)
   화면 스냅샷(exportSnapshot) + 지난번 선택 + 기본값
        ▼
설정 화면 ── 값이 바뀔 때마다 refreshForm (범위 설명, 요약 줄, 내보내기 버튼 켜고 끄기)
        │  [Export]
        ▼
진행 화면 ── renderOffline(299) ─▶ 진행률 막대, 남은 시간 어림
        │       [Stop] 또는 Esc ─▶ 설정 화면으로 돌아감 (파일 없음)
        ▼
인코딩: 피크 ─▶ 정규화 배율 ─▶ encodeWav(297)
        ▼
완료 화면 ── 파일명, 길이, 크기, 피크, 걸린 시간 ── [Download] ─▶ 브라우저 다운로드
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer.js`의 `initExport()` 호출	명령 `file.export`를 등록합니다. 메뉴, 커맨드 팔레트, 단축키가 같은 명령을 부릅니다.
`sequencer-ui.js`의 ↓ 버튼	이전의 "Phase 3에서 구현" 안내를 `runCommand('file.export')`로 바꿨습니다.
`sequencer.js`의 `initShortcuts`	Ctrl+Shift+R을 브라우저의 강력 새로고침 대신 이 창으로 연결합니다.
`sequencer.js`의 `applyProjectInfo`	파일명에 쓸 프로젝트 이름을 `document.body.dataset.projectTitle`에 둡니다.

3. 코드

3.1 화면의 값 모으기 (`sequencer-audio.js`)

```
export function exportSnapshot() {
  const loop = getLoop();
  return {
    tracks: playbackTracks(),
    audioClips: collectAudioClips(),
    midiClips: collectMidiClips(),
    getInstruments: playbackInstruments,
    getEffects: playbackEffects,
    bpm: currentBpm(),
    tempoChanges: [],
    loop: loop.on && loop.length > 0
      ? { start: Math.round(loop.start * TICKS_PER_BAR), end: Math.round((loop.start + loop.length) * TI
CKS_PER_BAR) }
      : null,
    loadSampleBytes: loadSampleBytes,
  };
}
```

클립 정보에 `selected` 값이 추가되었습니다. 화면의 클립 요소에 `is-selected` 클래스가 있는지를 담습니다.

3.2 창의 값 읽기와 계산

```
function readOptions() {
  const checked = document.querySelector('input[name="ex-range"]:checked');
  return {
    mode: checked ? checked.value : 'all',
    fromBar: Number(document.getElementById('ex-from').value),
    toBar: Number(document.getElementById('ex-to').value),
    sampleRate: segmentValue('ex-rate', 'rate') || 44100,
    bitDepth: segmentValue('ex-depth', 'depth') || 16,
    normalize: document.getElementById('ex-norm').checked,
    normalizeDb: clampNumber(document.getElementById('ex-norm-db').value, -12, 0, -0.3),
    tailSec: clampNumber(document.getElementById('ex-tail').value, 0, MAX_TAIL_SECONDS, 0),
    fileName: sanitizeFileName(document.getElementById('ex-file').value.replace(/\.wav$/i, '')),
  };
}
```

```
function rangeFor(o) {
  return resolveExportRange({
    mode: o.mode, clips: allClips(), loop: snap.loop, selected: selectedClips(),
    fromBar: o.fromBar, toBar: o.toBar, ticksPerBar: TICKS_PER_BAR,
  });
}
```

```
function describeRange(range) {
  const a = Math.floor(range.startTick / TICKS_PER_BAR) + 1;
  const b = Math.max(a, Math.ceil(range.endTick / TICKS_PER_BAR));
  return 'bar ' + a + ' – ' + b + ' · ' + formatDuration(rangeSeconds(range.startTick, range.endTick, sn
```

```
ap.bpm, snap.tempoChanges));
}
```

```
function computeAutoTail() {
  const effects = [];
  const layers = [];
  snap.tracks.forEach(function (t) {
    (snap.getEffects(t.track_id) || []).forEach(function (e) { effects.push(e); });
    (snap.getInstruments(t.track_id) || []).forEach(function (l) { layers.push(l); });
  });
  return autoTailSeconds(effects, layers, snap.bpm);
}
```

3.3 설정 화면 갱신

```
function refreshForm() {
  if (!snap) return;
  const o = readOptions();

  // 범위별 설명과 사용 가능 여부
  const all = rangeFor({ mode: 'all', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-all').textContent = all.ok ? describeRange(all) : 'Nothing to export y
et';
  const loop = rangeFor({ mode: 'loop', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-loop').textContent = loop.ok ? describeRange(loop) : 'No loop region';
  const sel = rangeFor({ mode: 'selected', fromBar: o.fromBar, toBar: o.toBar });
  const selCount = selectedClips().length;
  document.getElementById('ex-len-selected').textContent = sel.ok ? selCount + (selCount === 1 ? ' clip
· ' : ' clips · ') + describeRange(sel) : 'Select clips on the timeline first';
  const custom = rangeFor({ mode: 'custom', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-custom').textContent = custom.ok ? describeRange(custom) : '—';

  const loopRadio = document.querySelector('input[name="ex-range"][value="loop"]');
  const selRadio = document.querySelector('input[name="ex-range"][value="selected"]');
  loopRadio.disabled = !loop.ok;
  selRadio.disabled = !sel.ok;
  document.getElementById('ex-opt-loop').classList.toggle('is-off', !loop.ok);
  document.getElementById('ex-opt-selected').classList.toggle('is-off', !sel.ok);
  if ((o.mode === 'loop' && !loop.ok) || (o.mode === 'selected' && !sel.ok)) {
    document.querySelector('input[name="ex-range"][value="all"]').checked = true;
    o.mode = 'all';
  }
  document.getElementById('ex-from').disabled = o.mode !== 'custom';
  document.getElementById('ex-to').disabled = o.mode !== 'custom';
  document.getElementById('ex-custom').classList.toggle('is-off', o.mode !== 'custom');
  document.getElementById('ex-selected-note').hidden = o.mode !== 'selected';

  // 비트 깊이 안내와 정규화 칸
  document.getElementById('ex-depth-note').textContent = o.bitDepth === 16
    ? '16-bit adds a little dither noise so quiet tails stay smooth.'
    : (o.bitDepth === 24 ? '24-bit keeps more detail for further mixing.' : '32-bit float cannot clip in
side the file and is best for further editing.');
  document.getElementById('ex-norm-db').disabled = !o.normalize;

  // 요약 줄과 내보내기 버튼
```

```
  const range = rangeFor(o);
  const msg = document.getElementById('ex-msg');
  if (range.ok) {
    const seconds = rangeSeconds(range.startTick, range.endTick, snap.bpm, snap.tempoChanges) + o.tailSe
c;
    const frames = Math.ceil(seconds * o.sampleRate);
    document.getElementById('ex-sum-len').textContent = formatDuration(seconds);
    document.getElementById('ex-sum-fmt').textContent = 'WAV · ' + (o.sampleRate / 1000) + ' kHz · ' +
(o.bitDepth === 32 ? '32-bit float' : o.bitDepth + '-bit') + ' stereo';
    document.getElementById('ex-sum-size').textContent = '≈ ' + formatBytes(wavByteLength(frames, 2, o.b
itDepth));
    msg.hidden = true;
  } else {
    document.getElementById('ex-sum-len').textContent = '—';
    document.getElementById('ex-sum-fmt').textContent = '—';
    document.getElementById('ex-sum-size').textContent = '—';
    msg.textContent = reasonText(range.reason);
    msg.hidden = false;
  }
  document.getElementById('ex-go').disabled = !range.ok;
}
```

3.4 열기

```
export function openExport() {
  const dlg = document.getElementById('export-modal');
  if (!dlg || dlg.open) return;
  snap = exportSnapshot();
  result = null;
  rendering = false;
  cancelFlag = false;

  const prefs = loadPrefs();
  const projectRate = Number(document.body.dataset.sampleRate) === 48000 ? 48000 : 44100;
  pickSegment('ex-rate', 'rate', prefs.sampleRate === 48000 || prefs.sampleRate === 44100 ? prefs.sample
Rate : projectRate);
  pickSegment('ex-depth', 'depth', [16, 24, 32].indexOf(prefs.bitDepth) >= 0 ? prefs.bitDepth : 16);
  document.getElementById('ex-norm').checked = !!prefs.normalize;
  document.getElementById('ex-norm-db').value = String(clampNumber(prefs.normalizeDb, -12, 0, -0.3));

  document.querySelector('input[name="ex-range"][value="all"]').checked = true;
  const all = rangeFor({ mode: 'all' });
  document.getElementById('ex-from').value = '1';
  document.getElementById('ex-to').value = String(all.ok ? Math.max(1, Math.ceil(all.endTick / TICKS_PER
_BAR)) : 1);
  document.getElementById('ex-tail').value = String(computeAutoTail());
  document.getElementById('ex-file').value = exportFileName(document.body.dataset.projectTitle, new Date
());

  showView('form');
  refreshForm();
  dlg.showModal();
}
```

3.5 내보내기, 중지, 다운로드, 닫기

```
async function startExport() {
  const o = readOptions();
  const range = rangeFor(o);
  if (!range.ok || rendering) return;
  savePrefs(o);
  rendering = true;
  cancelFlag = false;
  startedAt = Date.now();
  document.getElementById('ex-stop').disabled = false;
  document.getElementById('ex-prog-title').textContent = 'Rendering…';
  document.getElementById('ex-prog-note').textContent = 'Keep this window open. You can stop it at any t
ime.';
  setProgress(0);
  showView('progress');

  let audio = snap.audioClips;
  let midi = snap.midiClips;
  if (o.mode === 'selected') {
    audio = audio.filter(function (c) { return c.selected; });
    midi = midi.filter(function (c) { return c.selected; });
  }

  try {
    const out = await renderOffline({
      tracks: snap.tracks, audioClips: audio, midiClips: midi,
      getInstruments: snap.getInstruments, getEffects: snap.getEffects,
      bpm: snap.bpm, tempoChanges: snap.tempoChanges,
      startTick: range.startTick, endTick: range.endTick, tailSec: o.tailSec, sampleRate: o.sampleRate,
      loadSampleBytes: snap.loadSampleBytes,
      onProgress: setProgress,
      isCancelled: function () { return cancelFlag; },
    });
    document.getElementById('ex-prog-title').textContent = 'Writing the file…';
    await nextFrame();
    const peak = peakOfChannels(out.channels);
    const gain = o.normalize ? normalizeGain(peak, o.normalizeDb) : 1;
    const bytes = encodeWav(out.channels, out.sampleRate, o.bitDepth, { gain: gain, dither: true });
    result = { bytes: bytes, fileName: o.fileName };

    document.getElementById('ex-done-name').textContent = o.fileName + '.wav';
    document.getElementById('ex-done-meta').textContent = formatDuration(out.frames / out.sampleRate) +
' · ' + formatBytes(bytes.length);
    const seconds = Math.max(1, Math.round((Date.now() - startedAt) / 1000));
    if (peak > 0) {
      const peakDb = 20 * Math.log10(peak * gain);
      document.getElementById('ex-done-note').textContent = 'Peak ' + peakDb.toFixed(1) + ' dBFS · rende
red in ' + seconds + (seconds === 1 ? ' second' : ' seconds')
        + (peak * gain > 1 ? ' · The mix is louder than 0 dBFS and may clip (turn on Normalize or lower
the master).' : '');
    } else {
      document.getElementById('ex-done-note').textContent = 'No sound was rendered. Check that the track
s are not muted and that clips are inside the range.';
    }
    showView('done');
```

```
  } catch (err) {
    showView('form');
    refreshForm();
    if (!(err && err.message === 'cancelled')) {
      const msg = document.getElementById('ex-msg');
      msg.textContent = 'Export failed: ' + (err && err.message ? err.message : 'unknown error');
      msg.hidden = false;
      if (window.console) console.warn('[export] 렌더링에 실패했습니다', err);
    }
  } finally {
    rendering = false;
  }
}
```

```
function stopExport() {
  if (!rendering) return;
  cancelFlag = true;
  document.getElementById('ex-stop').disabled = true;
  document.getElementById('ex-prog-title').textContent = 'Stopping…';
}
```

```
function downloadResult() {
  if (!result) return;
  const blob = new Blob([result.bytes], { type: 'audio/wav' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = result.fileName + '.wav';
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(function () { URL.revokeObjectURL(url); }, 60000);
}
```

```
export function closeExport() {
  const dlg = document.getElementById('export-modal');
  if (rendering) stopExport();
  if (dlg && dlg.open) dlg.close();
  result = null;
}
```

3.6 연결

```
export function initExport() {
  const dlg = document.getElementById('export-modal');
  if (!dlg) return;
  registerCommand('file.export', openExport, { label: 'Export audio…', group: 'file', keys: 'Ctrl+Shift+
R' });

  dlg.addEventListener('click', function (e) {
    const rate = e.target.closest('#ex-rate button[data-rate]');
    const depth = e.target.closest('#ex-depth button[data-depth]');
    if (rate) { pickSegment('ex-rate', 'rate', rate.dataset.rate); refreshForm(); }
    if (depth) { pickSegment('ex-depth', 'depth', depth.dataset.depth); refreshForm(); }
```

```
  });
  dlg.addEventListener('input', refreshForm);
  dlg.addEventListener('change', refreshForm);
  document.getElementById('ex-tail-auto').addEventListener('click', function () {
    if (!snap) return;
    document.getElementById('ex-tail').value = String(computeAutoTail());
    refreshForm();
  });
  document.getElementById('ex-go').addEventListener('click', startExport);
  document.getElementById('ex-stop').addEventListener('click', stopExport);
  document.getElementById('ex-download').addEventListener('click', downloadResult);
  document.getElementById('ex-cancel').addEventListener('click', closeExport);
  document.getElementById('ex-close').addEventListener('click', closeExport);
  document.getElementById('ex-x').addEventListener('click', closeExport);
  dlg.addEventListener('cancel', function (e) {
    if (rendering) { e.preventDefault(); stopExport(); }
  });
}
```

4. 화면의 구성

영역	내용
Format	WAV만 선택할 수 있습니다. MP3와 FLAC은 꺼 두고 "Phase 3에서 제공"이라고 알립니다.
Range	전체, 루프 구간, 선택한 클립, 직접 지정. 쓸 수 없는 항목은 흐리게 하고 이유를 오른쪽에 씁니다("No loop region", "Select clips on the timeline first").
Sample rate	44.1kHz, 48kHz. 처음에는 프로젝트의 샘플레이트를 따릅니다.
Bit depth	16bit, 24bit, 32bit float. 선택에 따라 아래 안내 문구가 바뀝니다.
MP3 bitrate	128, 192, 320을 꺼 둡니다. MP3 인코딩은 Phase 3에서 켭니다.
Normalize	켜면 목표 피크(기본 -0.3dBFS)를 입력할 수 있습니다.
Tail	초 단위 입력(0에서 30). Auto 버튼은 이펙트와 악기가 남기는 길이를 계산해 채웁니다.
File name	`{프로젝트명}_{YYYYMMDD_HHmm}`. 금지 글자는 `_`로 바뀌고, `.wav`를 직접 써도 중복되지 않습니다.
요약 줄	길이, 형식, 예상 크기. 값이 바뀔 때마다 다시 계산합니다.

5. 최신 문법으로 쓰면

```
// loadPrefs 와 clampNumber 를 최신 문법으로 쓴 모양
const loadPrefs = () => {
  try { return JSON.parse(localStorage.getItem(PREFS_KEY)) ?? {}; } catch { return {}; }
};
const clampNumber = (value, min, max, fallback) =>
  Number.isFinite(Number(value)) ? Math.min(max, Math.max(min, Number(value))) : fallback;

// 클립 선택 표시와 필터
const selectedClips = () => allClips().filter((c) => c.selected);
const pickSegment = (groupId, attr, value) =>
  document.getElementById(groupId).querySelectorAll('button').forEach((b) => { /* ... */ });
```

전통 문법 (실제 코드)	최신 문법
`raw ? JSON.parse(raw) : {}`	`JSON.parse(raw) ?? {}`
`catch (e) { ... }`	변수 없는 `catch { ... }`
`Array.prototype.forEach.call(목록, ...)`	`목록.forEach(...)` 또는 `[...목록].forEach`
`isFinite(n)`	`Number.isFinite(n)`

6. 왜 이렇게

선택	이유
창을 열 때 화면의 스냅샷을 찍습니다	창은 모달이라 열려 있는 동안 편집할 수 없습니다. 한 번 찍은 값을 계속 쓰면 값이 중간에 바뀌어 요약 줄과 결과가 어긋나는 일이 없습니다.
범위는 늘 "전체"로 시작합니다	클립이 선택된 채로 창을 열면 일부만 내보내기가 되기 쉽습니다. 모르고 곡의 한 조각만 받는 일을 막으려고, 선택한 클립은 사용자가 직접 고르게 했습니다.
"선택한 클립"은 타임라인에서 선택한 클립입니다	FL Studio의 Pattern 모드와 Ableton의 선택한 트랙처럼 "지금 고른 것만 내보내기"를 지원하려는 것입니다. 선택하지 않은 클립은 소리를 끄고, 트랙 이펙트와 믹서는 그대로 적용합니다.
화면의 클립 요소에서 선택 여부를 읽습니다	선택은 화면에만 있는 상태(`is-selected`)입니다. 재생에 넘기는 값을 모으는 함수(`collectAudioClips`, `collectMidiClips`)에 `selected` 한 줄만 더해, 같은 스냅샷을 재생과 내보내기가 함께 씁니다.
쓸 수 없는 범위를 숨기지 않고 흐리게 둡니다	사용자가 그런 범위가 있다는 것을 알고, 어떻게 쓰는지 안내문으로 알 수 있습니다. 쓸 수 없는 상태에서 선택되어 있었다면 자동으로 "전체"로 돌립니다.
샘플레이트, 비트 깊이, 정규화는 기억합니다	사용자는 보통 늘 같은 설정으로 내보냅니다. 범위, 꼬리, 파일명은 곡마다 다르므로 기억하지 않고 새로 정합니다. 브라우저 저장소를 못 쓰는 환경에서는 기본값으로 시작합니다.
처음 샘플레이트는 프로젝트의 값을 따릅니다	프로젝트가 48kHz인데 기억이 없으면 44.1kHz로 내보내서 소리를 다시 맞추는 일이 없게 합니다.
진행 화면의 안내는 "남은 시간"을 어림해서 보여 줍니다	10%가 지난 뒤에만 계산합니다. 처음에는 샘플을 불러오는 시간이 섞여서 값이 크게 틀리기 때문입니다.
렌더링 중 Esc는 닫지 않고 중지로 받습니다	바로 닫으면 렌더링이 눈에 보이지 않는 곳에서 계속 돕니다. 먼저 멈추고 설정 화면으로 돌아가게 합니다.
완료되어도 자동으로 내려받지 않습니다	결과를 확인(파일명, 피크, 소리가 비었는지)한 뒤 사용자가 Download를 누르게 합니다. PRD(FR-307)도 완료 시 다운로드 버튼을 둡니다.
소리가 비었으면 완료 화면이 알려 줍니다	뮤트했거나 범위에 클립이 없으면 무음 파일이 나옵니다. 그냥 두면 받은 파일을 열어 본 뒤에야 알게 됩니다.
피크가 0dBFS를 넘으면 경고를 붙입니다	정수 WAV는 그 값에서 잘려 소리가 거칠어집니다. 정규화를 켜거나 마스터를 낮추라고 안내합니다.
다운로드는 `Blob`과 임시 주소로 합니다	서버를 거치지 않아도 되고(클라이언트 렌더링, FR-302), 파일이 서버에 남지 않습니다. 임시 주소는 1분 뒤에 풉니다.
Ctrl+Shift+R을 가로챕니다	PRD와 메뉴의 단축키 표시가 Ctrl+Shift+R입니다. 시퀀서 화면은 작업 화면이어서 강력 새로고침보다 내보내기가 더 자주 쓰입니다. 입력칸에 커서가 있거나 다른 창이 열려 있으면 무시합니다.

7. 시험

- `docs/handoff/s1-11b_tests/ui_export.mjs` (46건)는 실제 시퀀서 화면(로그인한 시험 계정)에서 확인합니다. 시험용 트랙에 8초 사인파 클립 두 개(1에서 2마디, 5에서 6마디)를 놓고 시작하며, 내려받기를 가로채서 바이트를 페이지 안에서 `decodeAudioData`로 다시 읽습니다.

    - 창 열기: ↓ 버튼, FILE 메뉴(실제 마우스 클릭), Ctrl+Shift+R이 창을 엽니다. ✕, Cancel, Close가 닫습니다.
    - 기본 화면: 범위는 전체(`bar 1 – 6`), 루프와 선택 항목은 꺼져 있고, 파일명은 `{프로젝트명}_{YYYYMMDD_HHmm}`, 형식은 16bit이며 샘플레이트는 프로젝트를 따릅니다.
    - 기본값 내보내기: 파일이 RIFF, PCM, 스테레오, 16bit이고, 길이가 6마디이며, 1에서 2마디와 5에서 6마디에 소리(피크 0.4)가 있고 3에서 4마디는 무음입니다.
    - 옵션: 48kHz, 24bit, 정규화, 꼬리 1초가 파일에 반영됩니다. 피크가 0.966(-0.3dBFS)이고, 길이가 6마디 더하기 1초이며, 파일명의 `/`가 `_`로 바뀝니다.
    - 범위: 선택한 클립(길이 2마디), 루프 구간(길이 2마디), 직접 지정 3에서 4마디(빈 구간이라 길이 2마디에 완전한 무음, 완료 화면이 "No sound was rendered"라고 알림)가 맞습니다.
    - 잘못된 범위(끝이 시작보다 앞)는 내보내기 버튼이 꺼지고 안내가 뜹니다.
    - 중지: 진행 화면에서 Stop을 누르면 설정 화면으로 돌아오고 오류 안내가 없으며 파일이 만들어지지 않습니다. 렌더링 중 Esc는 창을 닫지 않고 "Stopping…"으로 바뀝니다.
    - 지난번에 고른 샘플레이트, 비트 깊이, 정규화를 기억합니다. 화면에서 JS 오류가 나지 않습니다.
- 시험 중 발견한 일 (모두 시험 쪽 문제였고 제품 코드는 그대로입니다):
    - 메뉴 항목은 프로그램이 만든 클릭(`isTrusted`가 아닌 클릭)을 무시합니다. 공용 명령 레지스트리가 같은 명령을 다시 부르는 무한 반복을 막으려는 기존 설계입니다. 시험이 실제 마우스 이벤트(`Input.dispatchMouseEvent`)로 메뉴를 누르도록 바꿨습니다.
    - 3번 단계에서 켠 정규화를 창이 기억해서, 다음 단계들의 피크가 0.4가 아니라 0.966으로 나왔습니다. 의도한 동작이므로 시험의 기대값을 바꾸고, 기억 기능의 확인을 겸하게 했습니다.
    - 시험 안의 여러 `const r` 선언이 한 번의 평가에서 충돌했습니다. 중괄호 블록으로 감쌌습니다.